

4. clear all the gradient of optimizers

`optimizer.zero_grad()`

`zero_grad()` ,

the new gradients are added to the existing ones, resulting in approximately double the values from the first pass. Calling `optimizer.zero_grad()` **resets the gradients, ensuring that the subsequent backward pass computes fresh gradients based only on the current batch, preventing unwanted accumulation**

We use it to prevent

- **Prevents gradient accumulation:** PyTorch gradients are accumulated in the `.grad` attribute of tensors. If `zero_grad()` is not called, the gradients from the current batch would be added to the gradients from all previous batches.
- **Ensure correct updates:** Accumulating gradients leads to incorrect parameter updates, as the optimizer would be stepping based on an average or sum of many different batches' gradients rather than just the current one.
- **Memory management:** Clearing the gradients helps manage memory usage more effectively by ensuring that only the current gradients are stored

`optimizer.zero_grad()` vs. `model.zero_grad()`

If the optimizer was initialized with all of the model's parameters (e.g., `optim = Adam(model.parameters(), lr=...)`),

then `optimizer.zero_grad()` and `model.zero_grad()` are functionally

equivalent. `optimizer.zero_grad()` is generally recommended as it is safer if you have parameters not belonging to a `nn.Module` or are using multiple optimizers for different parts of a model